LICEO RUSSELL · INFORMATICA · CLASSE PRIMA

Verifica di informatica

Teoria, scrittura e lettura di codice in Python

0 1 0 1 1 0 0 1

DURATA	PUNTEGGIO	TEORIA	SCRITTURA	LETTURA
80 minuti	110 punti	20 punti	60 punti	30 punti

NOME _____ COGNOME _____

CLASSE _____ DATA _____

Istruzioni per la consegna

- Va creato **un file per ciascun esercizio**. Il nome del file è quello indicato nell'intestazione dell'esercizio, trasformato in modo che sia **tutto minuscolo e senza spazi**: l'estensione deve essere `.py` per gli esercizi di programmazione e `.txt` per le risposte alle domande e agli snippet.

Parte	File da consegnare
Teoria	`domanda1.txt`, `domanda2.txt`, `domanda3.txt`
Scrittura	`esercizio1.py`, `esercizio2.py`, `esercizio3.py`
Lettura	`snippet1.txt`, `snippet2.txt`, `snippet3.txt`

- **Ogni** risposta va messa dentro ad una cartella nominata `cognome-nome`, **tutto minuscolo**. Se avete più nomi o più cognomi, metteteli tutti, concatenati dal carattere `-` (trattino). Ad esempio:

Vostro nome	Nome cartella
Marini Mattia	`marini-mattia`
Olaizola Elisa Maitea	`olaizola-elisa-maitea`

- Una volta finito, la cartella contenente i file va copiata in scrittura. Potete trovare la cartella in `File Explorer (icona cartella) -> Questo Pc -> Scrittura`.

Istruzioni

- La prova è divisa in tre parti: **teoria** (20 punti), **scrittura del codice** (60 punti) e **lettura del codice** (30 punti).
- Rispondi alle domande di teoria in modo **sintetico ma completo**, usando le righe a disposizione.
- Per la parte di scrittura scrivi i programmi in **Python**, uno per esercizio. Sono valutate la correttezza, la gestione dei casi limite e la chiarezza del codice.
- Nella parte di lettura spiega a parole che cosa fa ciascun frammento e riporta il suo output, senza riscrivere il codice.
- Non è consentito consultare appunti, libri o Internet. È possibile usare il **cheatsheet** messo su scrittura.

Riepilogo dei punteggi

Voce	Dettaglio	Punteggio
Teoria (20 punti)		
D1	CPU e GPU	**7**
D2	Caratteri e protocolli	**7**
D3	Il linguaggio assembly	**6**
Scrittura del codice (60 punti)		
E1	Conta le occorrenze	**20**
E2	Il valore è nella lista?	**15**
E3	Conta i numeri pari	**25**
Lettura del codice (30 punti)		
S1	Primo frammento	**10**
S2	Secondo frammento	**10**
S3	Terzo frammento	**10**
	Totale	**110 punti**

Il punteggio delle singole voci è riportato accanto a ciascuna domanda e a ciascun esercizio. La prova vale 110 punti: i primi 100 determinano il voto, i 10 successivi valgono la lode.

1. Teoria (20 punti)

Rispondi in modo breve e preciso, usando le righe sotto ogni domanda.

Domanda 1 — CPU e GPU — 7 punti

Descrivi brevemente la differenza principale fra **CPU** e **GPU**, spiegando le due diverse modalità con cui eseguono i calcoli. Indica anche un ambito in cui è più adatta ciascuna delle due.

> **Nota.** Una o due frasi per ciascuna delle due modalità sono sufficienti. Nomina le parole **sequenziale** e **parallela**.

Domanda 2 — Caratteri e protocolli — 7 punti

Che cos'è la **tabella ASCII**? Perché è necessario stabilire dei **protocolli**, cioè delle convenzioni condivise, per lo scambio di informazioni fra computer?

> **Nota.** Spiega come la tabella associ ad ogni carattere una sequenza di bit e fai un esempio. Per i protocolli, accenna al fatto che mittente e destinatario devono interpretare i dati allo stesso modo.

Domanda 3 **Il linguaggio assembly** **6 punti**

Che cos'è il linguaggio **assembly**? A che cosa serve e in che rapporto sta con il codice macchina?

Nota. Accenna al fatto che ogni istruzione assembly corrisponde a un'istruzione macchina e che viene tradotto da un programma chiamato **assembler**.

2. Scrittura del codice (60 punti)

Per ciascun esercizio scrivi un file separato (`esercizio1.py`, `esercizio2.py`, `esercizio3.py`), eseguilo e verifica il risultato con gli esempi. Il programma deve stampare **soltanto** il risultato richiesto e non deve chiedere dati con `input()`: usa le variabili già presenti nel template.

Prima di consegnare

- Il programma stampa **solo** il risultato richiesto, senza messaggi in più?
- Hai provato i casi limite: lista vuota, valore assente, numeri negativi?
- I nomi delle variabili sono chiari e il codice è rientrato in modo corretto?
- Ogni esercizio è nel suo file `esercizio1.py`, `esercizio2.py`, `esercizio3.py`?

È **fondamentale** che il programma funzioni per qualsiasi input gli venga dato. Gli esercizi strutturati per funzionare solo su un input specifico verranno valutati **0 punti**. Ad esempio, se venisse chiesto di contare le occorrenze di `a` nella lista `l`, il seguente programma varrebbe zero punti:

```
l = [4, 7, 4, 1, 4]
a = 4
print(3)
```

Esercizio 1 **Conta le occorrenze** ★★☆☆☆ **20 punti**

Consegna. Date la lista `l` e il valore `a`, conta quante volte `a` compare in `l` e stampa un solo numero: il numero di occorrenze trovate. Se `a` non compare nella lista, stampa `0`. La lista può essere vuota.

Nota. Il programma deve funzionare con qualsiasi valore di `a` (anche una stringa) e con qualsiasi lista, senza modificarla. Non usare `l.count(a)`: il conteggio va fatto scorrendo la lista.

TEMPLATE DI PARTENZA

```
l = [4, 7, 4, 1, 4]
a = 4

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [], a = 3`	`0`	La lista è vuota: non ci sono occorrenze.
`l = [1, 2, 3], a = 5`	`0`	`5` non compare nella lista.
`l = [4, 7, 4, 1, 4], a = 4`	`3`	`4` compare tre volte.

Esercizio 2 — Il valore è nella lista? ★☆☆☆☆ 15 punti

Consegna. Date la lista `l` e il valore `a`, stampa `True` se `a` compare almeno una volta in `l`, altrimenti stampa `False`. Se `l` è vuota, stampa `False`. Il programma non deve modificare la lista.

> **Nota.** Deve essere stampato soltanto un valore booleano (`True` oppure `False`), non un messaggio. Attenzione: con una lista vuota il risultato è `False`, non `True`.

TEMPLATE DI PARTENZA

```
l = [3, 5, 7, 12]
a = 7

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = [], a = 7`	`False`	La lista è vuota, quindi `a` non può esserci.
`l = [3, 5, 7], a = 7`	`True`	`7` è l'ultimo elemento della lista.
`l = [3, 5, 7], a = 4`	`False`	`4` non compare nella lista.

Esercizio 3 — Conta i numeri pari ★★★☆☆ 25 punti

Consegna. Data una lista di interi `l`, conta quanti elementi sono pari e stampa un solo numero: il conteggio trovato. Se la lista è vuota, stampa `0`. La lista può contenere numeri negativi e lo zero.

> **Nota.** Un numero è pari se `x % 2 == 0`. In Python `-4 % 2` vale `0` e `-3 % 2` vale `1`, quindi lo stesso controllo funziona anche con i numeri negativi. Attenzione: anche `0` è pari.

TEMPLATE DI PARTENZA

```
l = [3, 8, 0, -4, 7, 10]

# scrivi qui il tuo codice
```

ESEMPI DI INPUT E OUTPUT

Input	Output	Spiegazione
`l = []`	`0`	La lista è vuota: non ci sono numeri pari.
`l = [1, 3, 5]`	`0`	Sono tutti dispari.
`l = [3, 8, 0, -4, 7, 10]`	`4`	I pari sono `8, 0, -4, 10`.

3. Lettura del codice (30 punti)

Per ciascun frammento di codice spiega a parole che cosa fa e, se il programma stampa qualcosa, riporta anche l'output. Non devi riscrivere il codice: va dimostrato di averne compreso l'intento, descrivendone il funzionamento.

Snippet 1 — Primo frammento

★★☆☆☆ **10 punti**

```
l = [3, -1, 0, 7, -5, 12]
i = 0
while i < len(l):
    if l[i] > 0:
        print(l[i])
    i = i + 1
```

Snippet 2 — Secondo frammento

★★☆☆☆ **10 punti**

Le variabili `a` e `b` rappresentano gli estremi di un intervallo. Cosa stampa il programma? A cosa serve il blocco `if a > b`?

```
l = [4, 6, 2, 9]
a = 10
b = 1
if a > b:
    t = a
    a = b
    b = t
i = 0
ok = True
while i < len(l):
    if l[i] < a or l[i] > b:
        ok = False
    i = i + 1
print(ok)
```

Snippet 3 Terzo frammento

★★☆☆☆ **10 punti**

```
a = 3
b = 4
risultato = 1
i = 0
while i < b:
    risultato = risultato * a
    i = i + 1
print(risultato)
```